# Striver A2Z DSA Sheet: Checklist

450 problems · Name: _____ · Start: _____

| | | |
|---|---|---|
| Learn the Basics | 29 | |
| Sorting Techniques | 7 | |
| Arrays | 40 | |
| Binary Search | 32 | |
| Strings (Basic and Medium) | 15 | |
| Linked List | 31 | |
| Recursion | 25 | |
| Bit Manipulation | 18 | |
| Stack and Queues | 30 | |

| | | |
|---|---|---|
| Sliding Window and Two Pointer | 12 | |
| Heaps | 17 | |
| Greedy Algorithms | 16 | |
| Binary Trees | 38 | |
| Binary Search Trees | 16 | |
| Graphs | 54 | |
| Dynamic Programming | 56 | |
| Tries | 7 | |
| Strings (Hard) | 7 | |

■ Fill the box when solved.

★ Star it if you needed help, and re-solve it later.

Write the date in the margin for spaced revision (next day, Sunday, +30 days).

## Step 1 **Learn the Basics** 29

### 1.1 Things to know in C++

- ☐ User input / output
- ☐ Data types
- ☐ If-else statements
- ☐ Switch statement
- ☐ Arrays and strings
- ☐ For loops
- ☐ While loops
- ☐ Functions (pass by value / by reference)
- ☐ Time and space complexity

### 1.2 Build-up logical thinking

- ☐ Patterns 1–22

### 1.3 Learn STL

- ☐ C++ STL (pair, vector, list, deque, stack, queue, priority_queue, set, multiset, unordered_set, map, multimap, unordered_map, algorithms)

### 1.4 Know basic maths

- ☐ Count digits
- ☐ Reverse a number
- ☐ Check palindrome
- ☐ GCD / HCF
- ☐ Armstrong number
- ☐ Print all divisors
- ☐ Check for prime

### 1.5 Learn basic recursion

- ☐ Understand recursion by printing something N times
- ☐ Print name N times
- ☐ Print 1 to N
- ☐ Print N to 1
- ☐ Sum of first N numbers
- ☐ Factorial of N
- ☐ Reverse an array
- ☐ Check if a string is a palindrome
- ☐ Fibonacci number

### 1.6 Learn basic hashing

- ☐ Hashing theory (counting frequencies of array elements)
- ☐ Find highest / lowest frequency element

## Step 2 **Sorting Techniques** 7

### 2.1 Sorting I

- ☐ Selection sort
- ☐ Bubble sort
- ☐ Insertion sort

### 2.2 Sorting II

- ☐ Merge sort
- ☐ Recursive bubble sort
- ☐ Recursive insertion sort
- ☐ Quick sort

## Step 3 **Arrays** 40

### 3.1 Easy

- ☐ Largest element in an array
- ☐ Second largest element (without sorting)
- ☐ Check if the array is sorted
- ☐ Remove duplicates from sorted array
- ☐ Left rotate an array by one place
- ☐ Left rotate an array by D places
- ☐ Move zeros to end
- ☐ Linear search
- ☐ Union of two sorted arrays
- ☐ Find missing number in an array
- ☐ Maximum consecutive ones
- ☐ Find the number that appears once
- ☐ Longest subarray with given sum K (positives)
- ☐ Longest subarray with given sum K (positives + negatives)

### 3.2 Medium

- ☐ Two Sum
- ☐ Sort an array of 0s, 1s and 2s
- ☐ Majority element (> n/2 times)
- ☐ Kadane's algorithm: maximum subarray sum
- ☐ Print subarray with maximum sum
- ☐ Stock buy and sell
- ☐ Rearrange array elements by sign
- ☐ Next permutation
- ☐ Leaders in an array
- ☐ Longest consecutive sequence
- ☐ Set matrix zeroes
- ☐ Rotate matrix by 90 degrees
- ☐ Print the matrix in spiral order
- ☐ Count subarrays with given sum K

### 3.3 Hard

- [ ] Pascal's triangle
- [ ] Majority element (> n/3 times)
- [ ] 3Sum
- [ ] 4Sum
- [ ] Largest subarray with 0 sum
- [ ] Count subarrays with given XOR K
- [ ] Merge overlapping intervals
- [ ] Merge two sorted arrays without extra space
- [ ] Find the repeating and missing number
- [ ] Count inversions
- [ ] Reverse pairs
- [ ] Maximum product subarray

## Step 4 **Binary Search** 32

### 4.1 BS on 1D arrays

- [ ] Binary search to find X in sorted array
- [ ] Implement lower bound
- [ ] Implement upper bound
- [ ] Search insert position
- [ ] Floor and ceil in sorted array
- [ ] First and last occurrence of X
- [ ] Count occurrences in sorted array
- [ ] Search in rotated sorted array I
- [ ] Search in rotated sorted array II (duplicates)
- [ ] Find minimum in rotated sorted array
- [ ] Find out how many times the array has been rotated
- [ ] Single element in a sorted array
- [ ] Find peak element

### 4.2 BS on answers

- [ ] Square root of a number using BS
- [ ] Nth root of a number
- [ ] Koko eating bananas
- [ ] Minimum days to make M bouquets
- [ ] Find the smallest divisor given a threshold
- [ ] Capacity to ship packages within D days
- [ ] Kth missing positive number
- [ ] Aggressive cows
- [ ] Book allocation problem
- [ ] Split array: largest sum
- [ ] Painter's partition
- [ ] Minimise maximum distance to gas station
- [ ] Median of two sorted arrays
- [ ] Kth element of two sorted arrays

### 4.3 BS on 2D arrays

- [ ] Find the row with maximum number of 1s
- [ ] Search in a 2D matrix
- [ ] Search in a row- and column-wise sorted matrix
- [ ] Find peak element II
- [ ] Matrix median

## Step 5 **Strings (Basic and Medium)** 15

### 5.1 Basic

- [ ] Remove outermost parentheses
- [ ] Reverse words in a given string
- [ ] Largest odd number in a string
- [ ] Longest common prefix
- [ ] Isomorphic strings
- [ ] Check whether one string is a rotation of another
- [ ] Check if two strings are anagrams
- [ ] Sort characters by frequency

### 5.2 Medium

- [ ] Maximum nesting depth of parentheses
- [ ] Roman to integer and integer to Roman
- [ ] Implement atoi
- [ ] Count number of substrings (with exactly K distinct characters)
- [ ] Longest palindromic substring
- [ ] Sum of beauty of all substrings
- [ ] Reverse every word in a string

## Step 6 **Linked List** 31

### 6.1 Learn 1D LL

- [ ] Introduction to LL (convert array to LL)
- [ ] Insert a node in LL
- [ ] Delete a node in LL
- [ ] Find the length of LL
- [ ] Search an element in LL

### 6.2 Learn doubly LL

- [ ] Introduction to DLL (convert array to DLL)
- [ ] Insert a node in DLL
- [ ] Delete a node in DLL
- [ ] Reverse a DLL

### 6.3 Medium problems of LL

- [ ] Middle of a LL (tortoise-hare)
- [ ] Reverse a LL (iterative)
- [ ] Reverse a LL (recursive)
- [ ] Detect a loop in LL
- [ ] Find the starting point of loop in LL
- [ ] Length of loop in LL
- [ ] Check if LL is a palindrome
- [ ] Segregate odd and even nodes in LL
- [ ] Remove Nth node from the end of LL
- [ ] Delete the middle node of LL
- [ ] Sort LL
- [ ] Sort a LL of 0s, 1s and 2s
- [ ] Find the intersection point of Y LL
- [ ] Add 1 to a number represented by LL
- [ ] Add two numbers in LL

### 6.4 Medium problems of DLL

- [ ] Delete all occurrences of a key in DLL
- [ ] Find pairs with given sum in DLL
- [ ] Remove duplicates from sorted DLL

### 6.5 Hard problems of LL

- [ ] Reverse LL in groups of size K
- [ ] Rotate a LL
- [ ] Flattening of LL
- [ ] Clone a LL with random and next pointer

## Step 7 **Recursion** 25

### 7.1 Get a strong hold

- [ ] Recursive implementation of atoi
- [ ] Pow(x, n)
- [ ] Count good numbers
- [ ] Sort a stack using recursion
- [ ] Reverse a stack using recursion

### 7.2 Subsequences pattern

- [ ] Generate all binary strings
- [ ] Generate parentheses
- [ ] Print all subsequences / power set
- [ ] Learn all patterns of subsequences (theory)
- [ ] Count all subsequences with sum K
- [ ] Check if there exists a subsequence with sum K
- [ ] Combination sum
- [ ] Combination sum II
- [ ] Subset sum I
- [ ] Subset sum II
- [ ] Combination sum III
- [ ] Letter combinations of a phone number

### 7.3 Trying out all combos (hard)

- [ ] Palindrome partitioning
- [ ] Word search
- [ ] N-Queens
- [ ] Rat in a maze
- [ ] Word break
- [ ] M-coloring problem
- [ ] Sudoku solver
- [ ] Expression add operators

## Step 8 **Bit Manipulation** 18

### 8.1 Learn bit manipulation

- [ ] Introduction to bit manipulation (theory)
- [ ] Check if the ith bit is set
- [ ] Check if a number is odd
- [ ] Check if a number is a power of 2
- [ ] Count the number of set bits
- [ ] Set / unset the rightmost unset bit
- [ ] Swap two numbers
- [ ] Divide two integers without multiplication or division

### 8.2 Interview problems

- [ ] Count number of bits to be flipped to convert A to B
- [ ] Find the number that appears an odd number of times
- [ ] Power set (using bits)
- [ ] Find XOR of numbers from L to R
- [ ] Find the two numbers appearing an odd number of times

### 8.3 Advanced maths

- [ ] Print prime factors of a number
- [ ] All divisors of a number
- [ ] Sieve of Eratosthenes
- [ ] Prime factorisation using sieve
- [ ] Power(n, x)

## Step 9 **Stack and Queues** 30

### 9.1 Learning

- [ ] Implement stack using arrays
- [ ] Implement queue using arrays
- [ ] Implement stack using queue
- [ ] Implement queue using stack
- [ ] Implement stack using LL
- [ ] Implement queue using LL
- [ ] Check for balanced parentheses
- [ ] Implement min stack

### 9.2 Prefix, infix, postfix conversions

- [ ] Infix to postfix
- [ ] Prefix to infix
- [ ] Prefix to postfix
- [ ] Postfix to prefix
- [ ] Postfix to infix
- [ ] Infix to prefix

### 9.3 Monotonic stack / queue

- [ ] Next greater element
- [ ] Next greater element II
- [ ] Next smaller element
- [ ] Number of NGEs to the right
- [ ] Trapping rainwater
- [ ] Sum of subarray minimums
- [ ] Asteroid collision
- [ ] Sum of subarray ranges
- [ ] Remove K digits
- [ ] Largest rectangle in histogram
- [ ] Maximal rectangle

### 9.4 Implementation problems

- [ ] Sliding window maximum
- [ ] Stock span problem
- [ ] The celebrity problem
- [ ] LRU cache
- [ ] LFU cache

## Step 10 **Sliding Window and Two Pointer** 12

### 10.1 Medium

- [ ] Longest substring without repeating characters
- [ ] Max consecutive ones III
- [ ] Fruit into baskets
- [ ] Longest repeating character replacement
- [ ] Binary subarrays with sum
- [ ] Count number of nice subarrays
- [ ] Number of substrings containing all three characters
- [ ] Maximum points you can obtain from cards

### 10.2 Hard

- [ ] Longest substring with at most K distinct characters
- [ ] Subarrays with K different integers
- [ ] Minimum window substring
- [ ] Minimum window subsequence

## Step 11 **Heaps** 17

### 11.1 Learning

- [ ] Introduction to priority queues using binary heaps
- [ ] Min heap and max heap implementation
- [ ] Check if an array represents a min-heap
- [ ] Convert min heap to max heap

### 11.2 Medium

- [ ] Kth largest element in an array
- [ ] Kth smallest element in an array
- [ ] Sort a K-sorted array
- [ ] Merge M sorted lists
- [ ] Replace each array element by its rank
- [ ] Task scheduler
- [ ] Hands of straights

### 11.3 Hard

- [ ] Design Twitter
- [ ] Connect N ropes with minimal cost
- [ ] Kth largest element in a stream
- [ ] Maximum sum combination
- [ ] Find median from data stream
- [ ] K most frequent elements

## Step 12 **Greedy Algorithms** 16

### 12.1 Easy

- [ ] Assign cookies
- [ ] Fractional knapsack
- [ ] Greedy algorithm to find minimum number of coins
- [ ] Lemonade change
- [ ] Valid parenthesis checker

### 12.2 Medium / hard

- [ ] N meetings in one room
- [ ] Jump game
- [ ] Jump game II
- [ ] Minimum number of platforms
- [ ] Job sequencing problem
- [ ] Candy
- [ ] Program for shortest job first (SJF) CPU scheduling
- [ ] Program for least recently used (LRU) page replacement
- [ ] Insert interval
- [ ] Merge intervals
- [ ] Non-overlapping intervals

## Step 13 **Binary Trees** 38

### 13.1 Traversals

- [ ] Introduction to trees
- [ ] Binary tree representation in C++
- [ ] Binary tree traversals (theory)
- [ ] Preorder traversal
- [ ] Inorder traversal
- [ ] Postorder traversal
- [ ] Level order traversal
- [ ] Iterative preorder traversal
- [ ] Iterative inorder traversal
- [ ] Postorder traversal (2 stacks)
- [ ] Postorder traversal (1 stack)
- [ ] Preorder, inorder and postorder in one traversal

### 13.2 Medium

- [ ] Height of a binary tree
- [ ] Check if the binary tree is height-balanced
- [ ] Diameter of binary tree
- [ ] Maximum path sum
- [ ] Check if two trees are identical
- [ ] Zigzag traversal
- [ ] Boundary traversal
- [ ] Vertical order traversal
- [ ] Top view of binary tree
- [ ] Bottom view of binary tree
- [ ] Right / left view of binary tree
- [ ] Symmetric binary tree

### 13.3 Hard

- [ ] Root to node path in binary tree
- [ ] LCA in binary tree
- [ ] Maximum width of binary tree
- [ ] Check for children sum property
- [ ] Print all nodes at distance K
- [ ] Minimum time taken to burn the binary tree from a node
- [ ] Count total nodes in a complete binary tree
- [ ] Requirements needed to construct a unique binary tree (theory)
- [ ] Construct binary tree from inorder and preorder
- [ ] Construct binary tree from inorder and postorder
- [ ] Serialize and deserialize binary tree
- [ ] Morris preorder traversal
- [ ] Morris inorder traversal
- [ ] Flatten binary tree to LL

## Step 14 **Binary Search Trees** 16

### 14.1 Concepts

- [ ] Introduction to BST
- [ ] Search in a BST

### 14.2 Practice problems

- [ ] Find min / max in BST
- [ ] Ceil in a BST
- [ ] Floor in a BST
- [ ] Insert a given node in BST
- [ ] Delete a node in BST
- [ ] Kth smallest / largest element in BST
- [ ] Check if a tree is a BST
- [ ] LCA in BST
- [ ] Construct a BST from a preorder traversal
- [ ] Inorder successor / predecessor in BST
- [ ] BST iterator
- [ ] Two sum in BST
- [ ] Recover BST (correct BST with two nodes swapped)
- [ ] Largest BST in binary tree

## Step 15 **Graphs** 54

### 15.1 Learning

- [ ] Graph and types
- [ ] Graph representation in C++
- [ ] Connected components
- [ ] BFS
- [ ] DFS

### 15.2 Problems on BFS / DFS

- [ ] Number of provinces
- [ ] Connected components in a matrix (number of islands)
- [ ] Flood fill
- [ ] Rotten oranges
- [ ] Detect a cycle in an undirected graph (BFS)
- [ ] Detect a cycle in an undirected graph (DFS)
- [ ] 0/1 matrix (distance of nearest cell)
- [ ] Surrounded regions (replace O's with X's)
- [ ] Number of enclaves
- [ ] Word ladder I
- [ ] Word ladder II
- [ ] Number of distinct islands
- [ ] Bipartite graph
- [ ] Detect cycle in a directed graph (DFS)

### 15.3 Topo sort and problems

- [ ] Topological sort (DFS)
- [ ] Kahn's algorithm (BFS)
- [ ] Detect cycle in a directed graph (BFS)
- [ ] Course schedule I
- [ ] Course schedule II
- [ ] Find eventual safe states
- [ ] Alien dictionary

### 15.4 Shortest path algorithms and problems

- [ ] Shortest path in undirected graph with unit weights
- [ ] Shortest path in DAG
- [ ] Dijkstra's algorithm (priority queue)
- [ ] Dijkstra's algorithm (set)
- [ ] Why priority queue and not queue in Dijkstra (theory)
- [ ] Print shortest path
- [ ] Shortest distance in a binary maze
- [ ] Path with minimum effort
- [ ] Cheapest flights within K stops
- [ ] Network delay time
- [ ] Number of ways to arrive at destination
- [ ] Minimum multiplications to reach end
- [ ] Bellman-Ford algorithm
- [ ] Floyd-Warshall algorithm
- [ ] Find the city with the smallest number of neighbours at a threshold distance

### 15.5 MST / disjoint set and problems

- [ ] Minimum spanning tree (theory)
- [ ] Prim's algorithm
- [ ] Disjoint set (union by rank / size, path compression)
- [ ] Kruskal's algorithm
- [ ] Number of operations to make network connected
- [ ] Most stones removed with same row or column
- [ ] Accounts merge
- [ ] Number of islands II
- [ ] Making a large island
- [ ] Swim in rising water

### 15.6 Other algorithms

- [ ] Bridges in graph (Tarjan)
- [ ] Articulation point in graph
- [ ] Kosaraju's algorithm (strongly connected components)

## Step 16 **Dynamic Programming** 56

### 16.1 Introduction to DP

- [ ] DP introduction (memoization vs tabulation)
- [ ] Climbing stairs
- [ ] Frog jump
- [ ] Frog jump with K distances
- [ ] Maximum sum of non-adjacent elements
- [ ] House robber

### 16.2 2D / 3D DP and DP on grids

- [ ] Ninja's training
- [ ] Grid unique paths
- [ ] Unique paths II
- [ ] Minimum path sum in grid
- [ ] Minimum path sum in triangular grid
- [ ] Minimum / maximum falling path sum
- [ ] 3D DP: Ninja and his friends (cherry pickup)

### 16.3 DP on subsequences

- [ ] Subset sum equal to target
- [ ] Partition equal subset sum
- [ ] Partition a set into two subsets with minimum absolute sum difference
- [ ] Count subsets with sum K
- [ ] Count partitions with given difference
- [ ] 0/1 knapsack
- [ ] Minimum coins
- [ ] Target sum
- [ ] Coin change II
- [ ] Unbounded knapsack
- [ ] Rod cutting problem

### 16.4 DP on strings

- [ ] Longest common subsequence
- [ ] Print longest common subsequence
- [ ] Longest common substring
- [ ] Longest palindromic subsequence
- [ ] Minimum insertions to make string palindrome
- [ ] Minimum insertions / deletions to convert string A to B
- [ ] Shortest common supersequence
- [ ] Distinct subsequences
- [ ] Edit distance
- [ ] Wildcard matching

### 16.5 DP on stocks

- [ ] Best time to buy and sell stock
- [ ] Buy and sell stock II
- [ ] Buy and sell stock III
- [ ] Buy and sell stock IV
- [ ] Buy and sell stock with cooldown
- [ ] Buy and sell stock with transaction fee

### 16.6 DP on LIS

- [ ] Longest increasing subsequence
- [ ] Print longest increasing subsequence
- [ ] LIS using binary search
- [ ] Largest divisible subset
- [ ] Longest string chain
- [ ] Longest bitonic subsequence
- [ ] Number of longest increasing subsequences

### 16.7 MCM / partition DP

- [ ] Matrix chain multiplication
- [ ] Matrix chain multiplication (bottom-up)
- [ ] Minimum cost to cut the stick
- [ ] Burst balloons
- [ ] Evaluate boolean expression to true
- [ ] Palindrome partitioning II
- [ ] Partition array for maximum sum

### 16.8 DP on squares

- [ ] Maximum rectangle area with all 1s
- [ ] Count square submatrices with all ones

## Step 17 **Tries** 7

### 17.1 Theory

- [ ] Implement trie (insert, search, startsWith)
- [ ] Implement trie II (count words / prefixes, erase)

### 17.2 Problems

- [ ] Longest string with all prefixes
- [ ] Number of distinct substrings in a string
- [ ] Power set (bit manipulation)
- [ ] Maximum XOR of two numbers in an array
- [ ] Maximum XOR with an element from array

## Step 18 **Strings (Hard)** 7

### 18.1 Hard problems

- [ ] Minimum number of bracket reversals to make an expression balanced
- [ ] Count and say
- [ ] Hashing in strings / Rabin-Karp
- [ ] Z-function
- [ ] KMP algorithm / LPS (prefix function) array
- [ ] Shortest palindrome
- [ ] Longest happy prefix